

5.1 Translation:

```
//Shafin Ahmed 232-15-184 65-M1
//Translation

#include <GL/glut.h>
#include <stdio.h>
#include <GL/gl.h>
float tx=0.00;
float ty=0.00;
void init(void)
{
    glClearColor(0.0,0.0,0.0,0.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-100.0, 100.0, -100.0, 100.0, -1.0, 1.0);
}
void Draw()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPushMatrix();
    glTranslatef(tx,ty,0);

    glColor3f(1,0,0);
    glRectf(-75,-75,75,75);

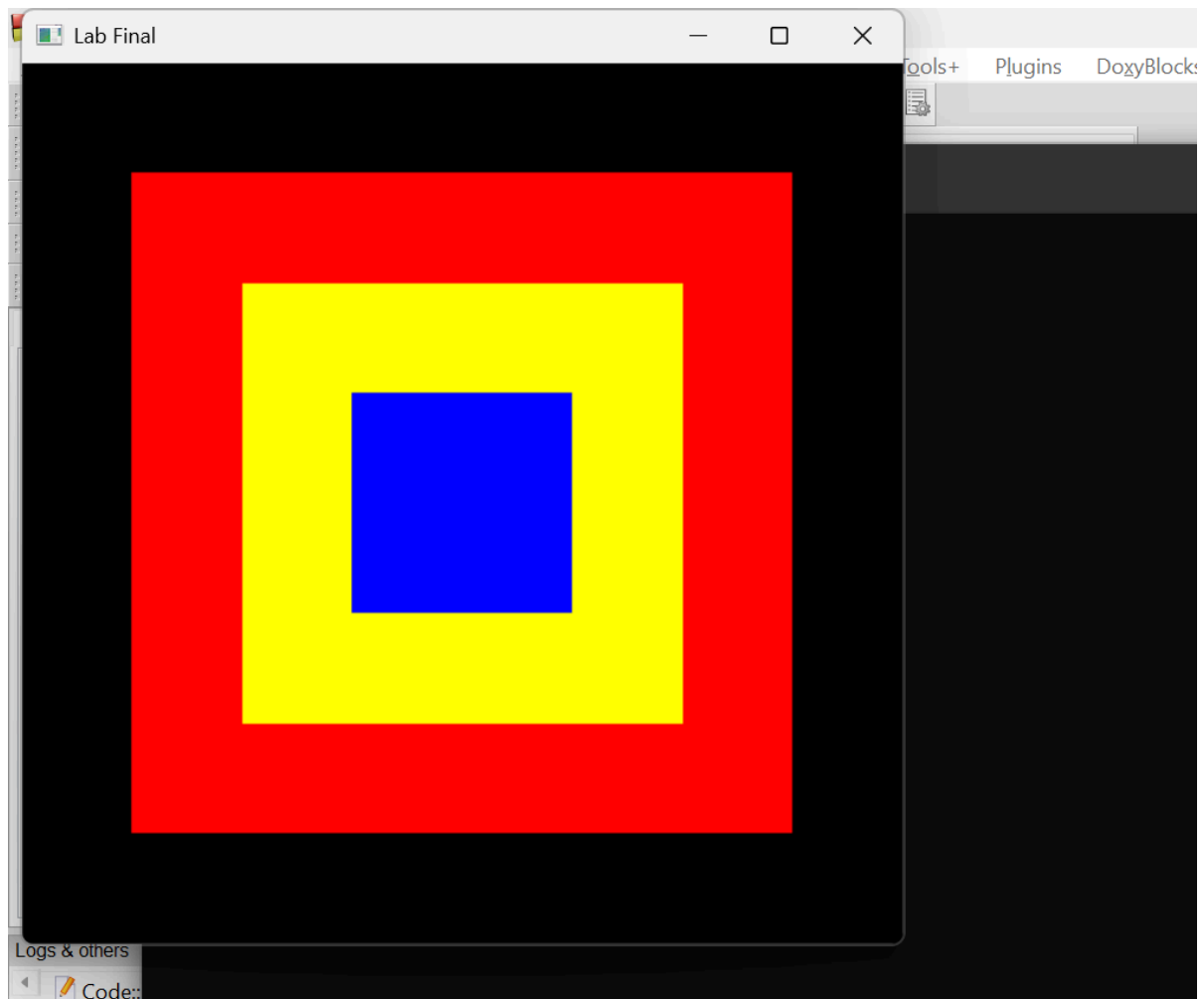
    glColor3f(1,1,0);
    glRectf(-50,-50,50,50);

    glColor3f(0,0,1);
    glRectf(-25,-25,25,25);

    glPopMatrix();
    glutSwapBuffers();
}
void spe_key(int key, int x, int y){
    switch(key){
        case GLUT_KEY_UP:
            ty=ty+5;
            glutPostRedisplay();
            break;
        case GLUT_KEY_DOWN:
            ty=ty-5;
            glutPostRedisplay();
            break;
        case GLUT_KEY_RIGHT:
            tx=tx+5;
            glutPostRedisplay();
            break;
        case GLUT_KEY_LEFT:
            tx=tx-5;
            glutPostRedisplay();
    }
}
```

```
        break;
    default:
        break;
    }
}
int main(int argc, char **argv){
    glutInit(&argc, argv);
    glutInitDisplayMode ( GLUT_RGB | GLUT_DOUBLE );
    glutInitWindowPosition(0,0);
    glutInitWindowSize(500,500);
    glutCreateWindow("Lab Final");
    init();
    glutDisplayFunc(Draw);
    glutSpecialFunc(spe_key);
    glutMainLoop();
    return 0;
}
```

Output:



5.2 Scaling

```
//Shafin Ahmed 232-15-184 65-M1
//Scaling
#include <GL/glut.h>
#include <stdio.h>
#include <GL/gl.h>
// Original coordinates
float x1 = 50, y1 = 50;
float x2 = 150, y2 = 50;
float x3 = 100, y3 = 150;

// Scaling factors
float sx, sy;

// Scaled coordinates
float sx1, sy1, sx2, sy2, sx3, sy3;

void init(void)
{
    glClearColor(0.0,0.0,0.0,0.0); //GLfloat red,green,blue,alpha initial value 0 alpha values
    used by glclear to clear the color buffers
    glMatrixMode(GL_PROJECTION); // To specify which matrix is the current matrix &
    projection applies subsequent matrix to projection matrix stack
    glLoadIdentity();
    glOrtho(0.0, 400.0, 0.0, 400.0, -1.0, 1.0);
    //gluOrtho2D(0.0,300.0,0.0,300.0); // Orthographic representation; multiply the current
    matrix by an orthographic matrix 2D= left right,bottom,top equivalent near=-1,far=1
}
void Draw()
{
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw Original Triangle (Blue)
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
        glVertex2f(x1, y1);
        glVertex2f(x2, y2);
        glVertex2f(x3, y3);
    glEnd();

    // Draw Scaled Triangle (Red)
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_TRIANGLES);
        glVertex2f(sx1, sy1);
        glVertex2f(sx2, sy2);
        glVertex2f(sx3, sy3);
    glEnd();

    glutSwapBuffers();
}
```

```

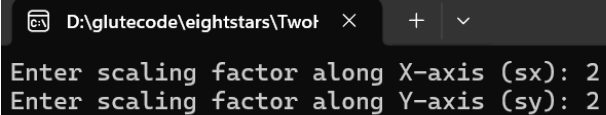
int main(int argc, char **argv){
    printf("Enter scaling factor along X-axis (sx): ");
    scanf("%f", &sx);
    printf("Enter scaling factor along Y-axis (sy): ");
    scanf("%f", &sy);

    // Apply Scaling Transformation
    sx1 = sx * x1;
    sy1 = sy * y1;
    sx2 = sx * x2;
    sy2 = sy * y2;
    sx3 = sx * x3;
    sy3 = sy * y3;

    glutInit(&argc, argv);
    glutInitDisplayMode ( GLUT_RGB | GLUT_DOUBLE );
    glutInitWindowPosition(0,0);
    glutInitWindowSize(500,500);
    glutCreateWindow("Lab Final");
    init();
    glutDisplayFunc(Draw);
    glutMainLoop();
    return 0;
}

```

Input:



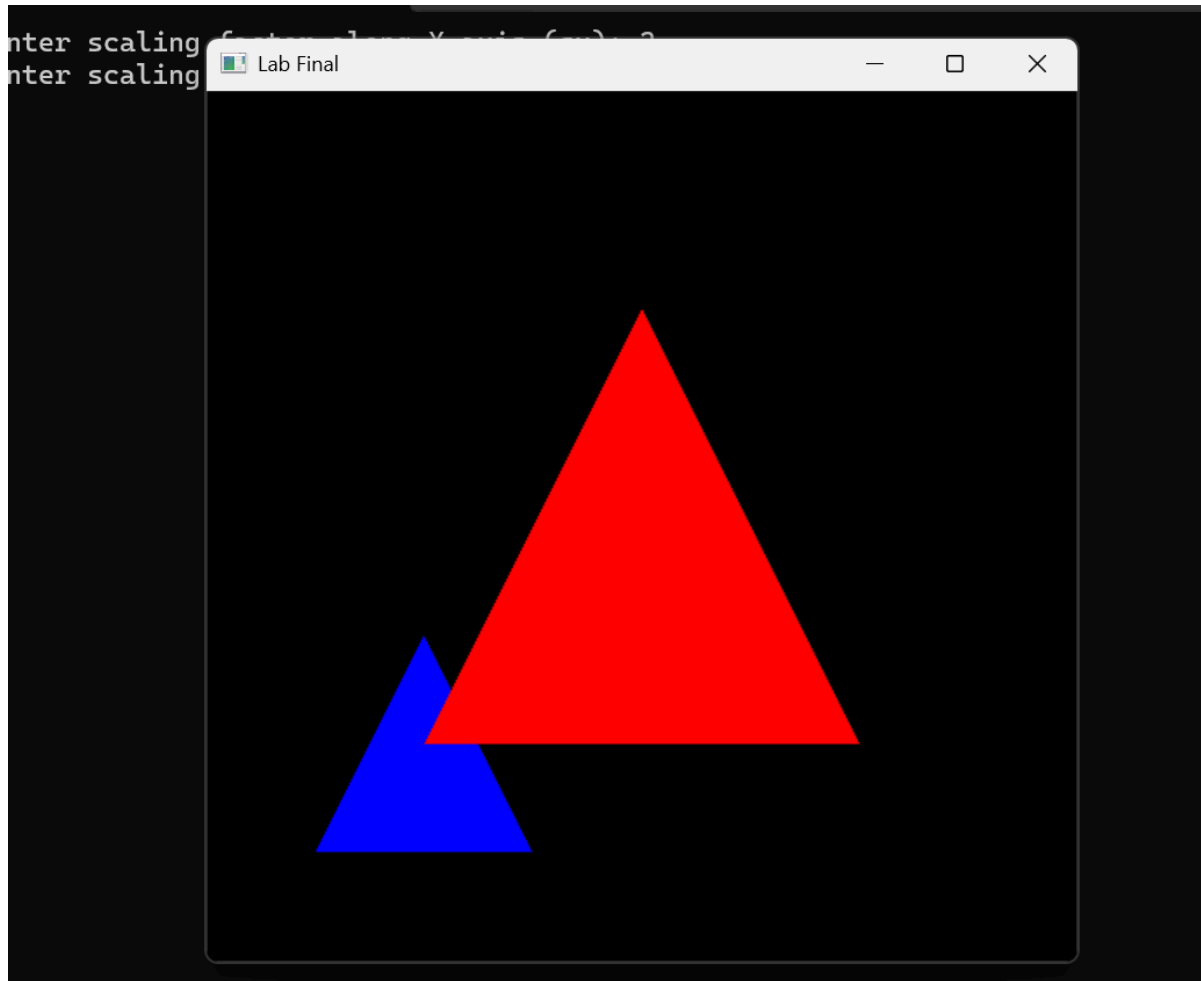
A screenshot of a terminal window with a dark background. The title bar shows the file path 'D:\glutocode\eightstars\Two' and standard window controls. The terminal displays two lines of text: 'Enter scaling factor along X-axis (sx): 2' and 'Enter scaling factor along Y-axis (sy): 2'.

```

D:\glutocode\eightstars\Two
Enter scaling factor along X-axis (sx): 2
Enter scaling factor along Y-axis (sy): 2

```

Output:



5.3 Reflection:

```
//Shafin Ahmed 232-15-184 65-M1
//Reflection
#include <GL/glut.h>
#include <stdio.h>
#include <GL/gl.h>
// Original triangle coordinates
float x1 = 50, y1 = 50;
float x2 = 150, y2 = 50;
float x3 = 100, y3 = 150;

// Reflected coordinates
float rx1, ry1, rx2, ry2, rx3, ry3;

int choice;

void init(void)
```

```

{
    glClearColor(0.0,0.0,0.0,0.0); //GLfloat red,green,blue,alpha initial value 0 alpha values
    used by glclear to clear the color buffers
    glMatrixMode(GL_PROJECTION); // To specify which matrix is the current matrix &
    projection applies subsequent matrix to projection matrix stack
    glLoadIdentity();
    glOrtho(-400.0, 400.0, -400.0, 400.0, -1.0, 1.0);
    //gluOrtho2D(0.0,300.0,0.0,300.0); // Orthographic representation; multiply the current
    matrix by an orthographic matrix 2D= left right,bottom,top equivalent near=-1,far=1
}

void Draw()
{
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw Original Triangle (Blue)
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
        glVertex2f(x1, y1);
        glVertex2f(x2, y2);
        glVertex2f(x3, y3);
    glEnd();

    // Draw Reflected Triangle (Red)
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_TRIANGLES);
        glVertex2f(rx1, ry1);
        glVertex2f(rx2, ry2);
        glVertex2f(rx3, ry3);
    glEnd();

    glutSwapBuffers();
}

int main(int argc, char **argv){
    printf("Choose Reflection Type:\n");
    printf("1. Reflection about X-axis\n");
    printf("2. Reflection about Y-axis\n");
    printf("3. Reflection about Diagonal (Origin)\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    if(choice == 1)
    {
        // Reflection about X-axis (x, y) -> (x, -y)
        rx1 = x1; ry1 = -y1;
        rx2 = x2; ry2 = -y2;
        rx3 = x3; ry3 = -y3;
    }
    else if(choice == 2)
    {
        // Reflection about Y-axis (x, y) -> (-x, y)
    }
}

```

```

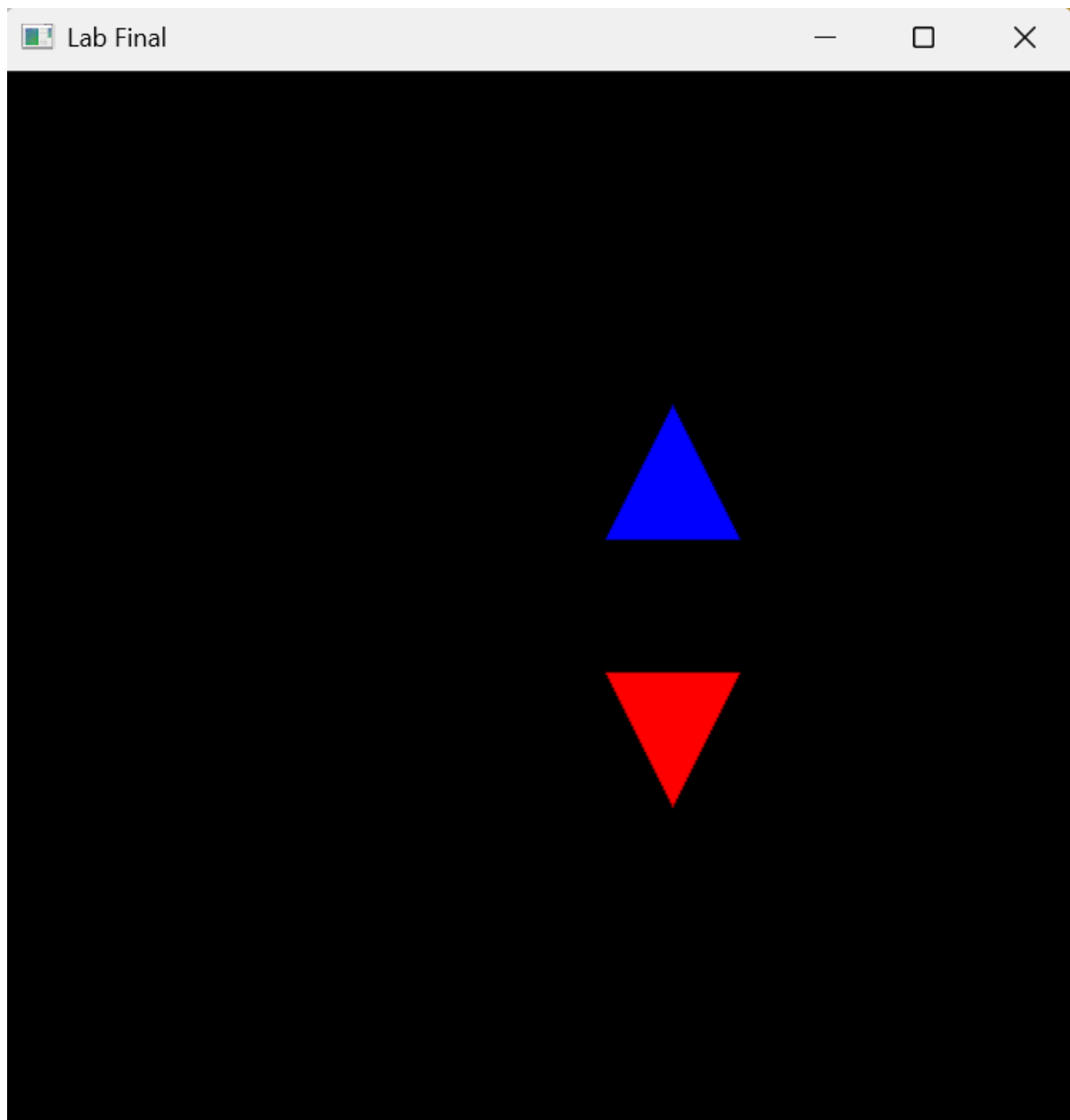
    rx1 = -x1; ry1 = y1;
    rx2 = -x2; ry2 = y2;
    rx3 = -x3; ry3 = y3;
}
else if(choice == 3)
{
    // Reflection about diagonal / origin (x, y) -> (-x, -y)
    rx1 = -x1; ry1 = -y1;
    rx2 = -x2; ry2 = -y2;
    rx3 = -x3; ry3 = -y3;
}
else
{
    printf("Invalid choice!");
    return 0;
}
glutInit(&argc,argv);
glutInitDisplayMode ( GLUT_RGB | GLUT_DOUBLE );
glutInitWindowPosition(0,0);
glutInitWindowSize(500,500);
glutCreateWindow("Lab Final");
init();
glutDisplayFunc(Draw);
glutMainLoop();
return 0;
}

```

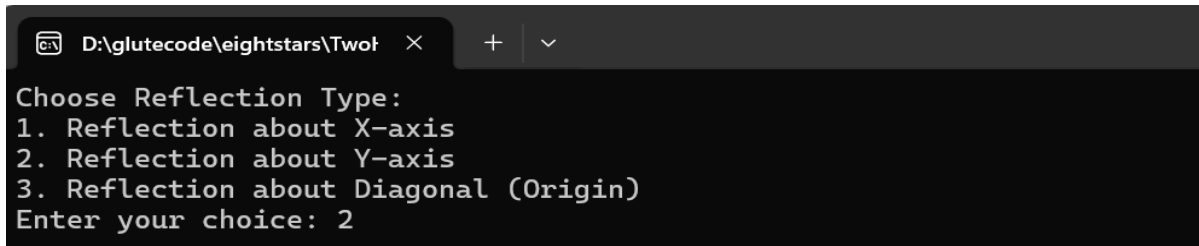
Input:

```
D:\glutecode\eightstars\Two1 x + v
Choose Reflection Type:
1. Reflection about X-axis
2. Reflection about Y-axis
3. Reflection about Diagonal (Origin)
Enter your choice: 1
```

Output:



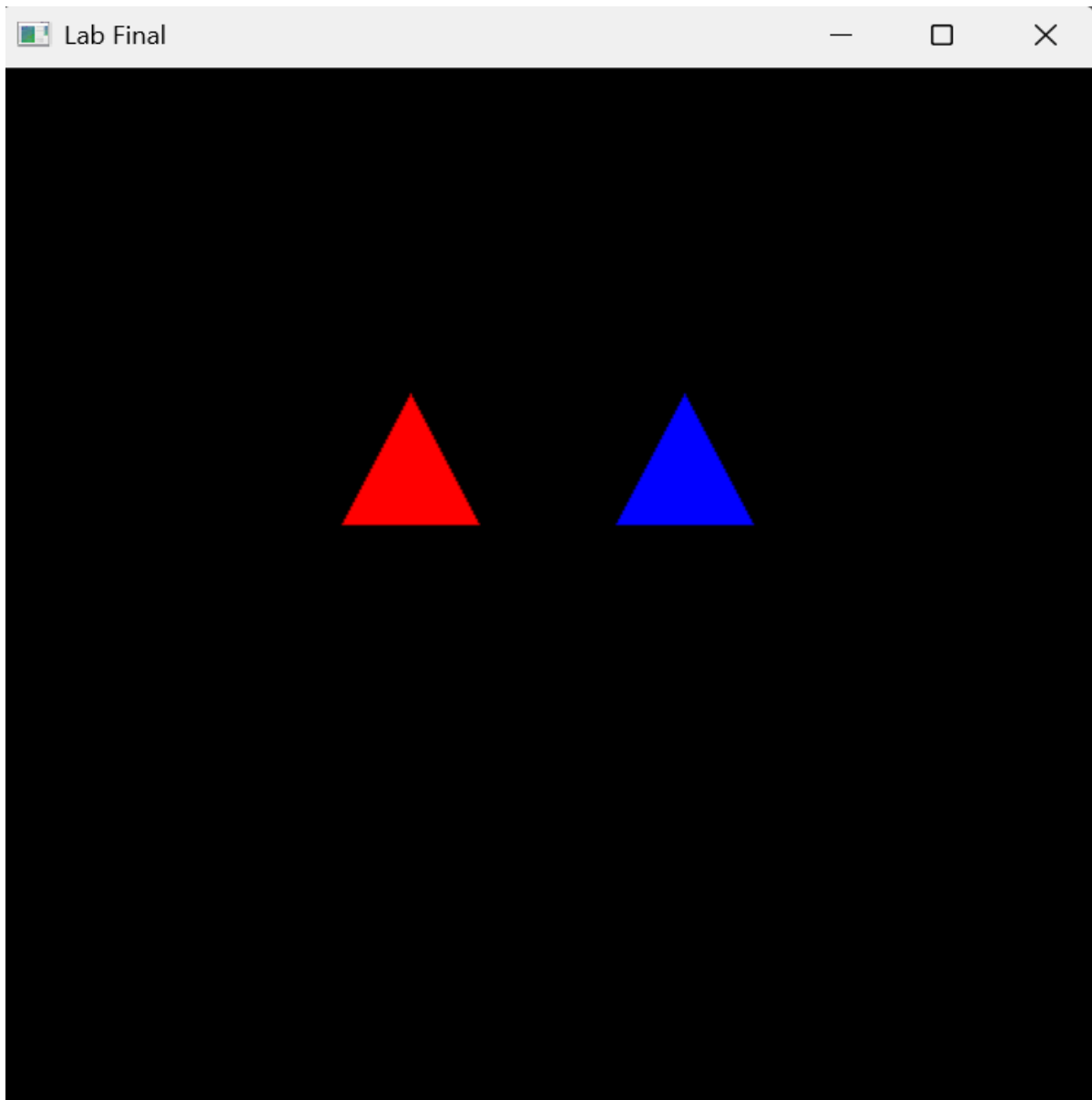
Input:



```
D:\glutecode\eightstars\Two1  X  +  v
Choose Reflection Type:
1. Reflection about X-axis
2. Reflection about Y-axis
3. Reflection about Diagonal (Origin)
Enter your choice: 2
```

A terminal window with a dark background. The title bar shows the file path 'D:\glutecode\eightstars\Two1' and standard window controls (close, maximize, minimize). The terminal content displays a menu for choosing a reflection type, with three options listed. The prompt 'Enter your choice:' is followed by the number '2'.

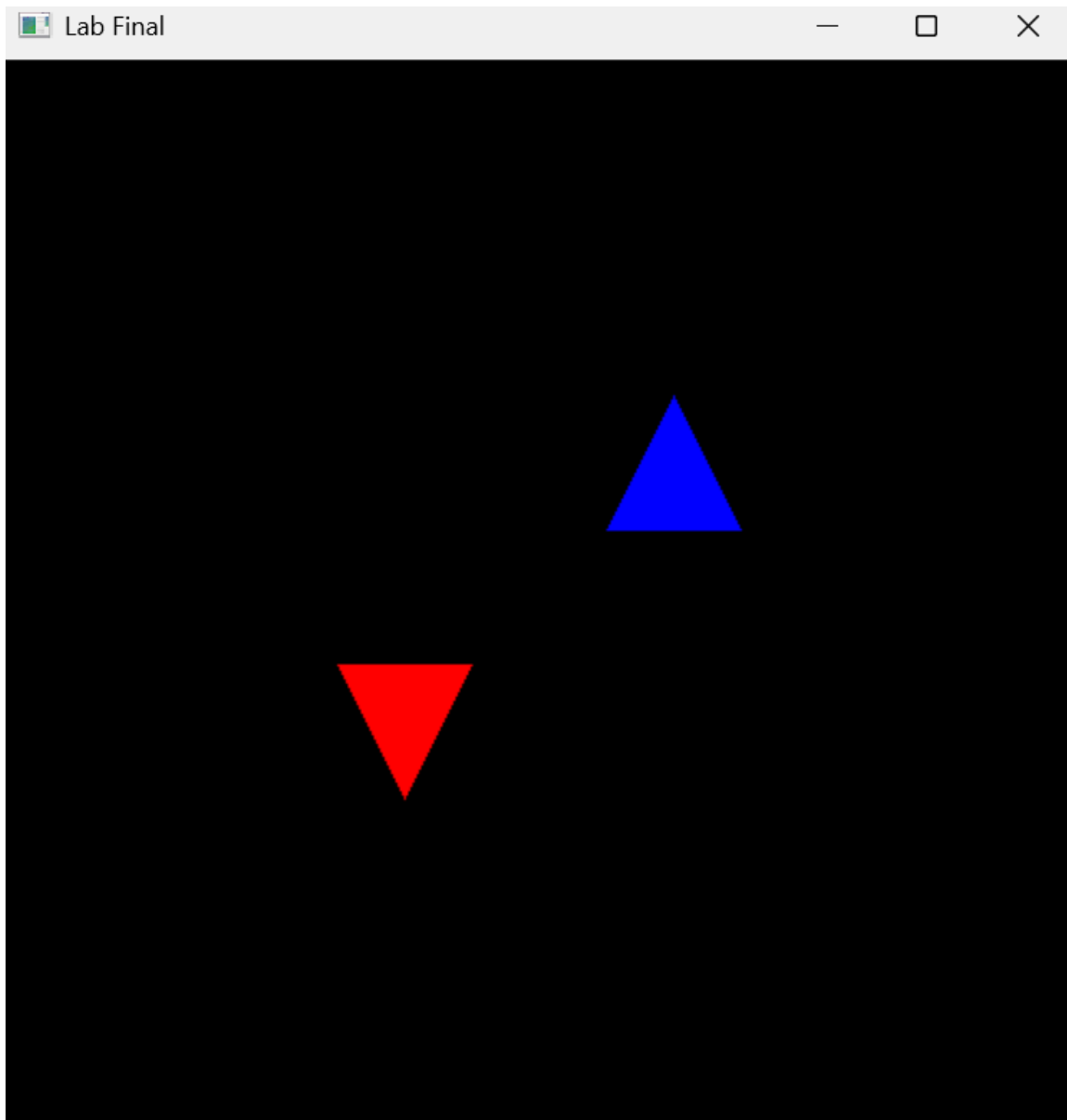
Output:



Input:

```
D:\glutecode\eightstars\TwoT  X + v
Choose Reflection Type:
1. Reflection about X-axis
2. Reflection about Y-axis
3. Reflection about Diagonal (Origin)
Enter your choice: 3
```

Output:



5.4 Shear:

```
//Shafin Ahmed 232-15-184 65-M1
//Shear
#include <GL/glut.h>
#include <stdio.h>
#include <GL/gl.h>
// Original triangle coordinates
float x1 = 50, y1 = 50;
float x2 = 150, y2 = 50;
```

```

float x3 = 100, y3 = 150;

// Shear factors
float shx = 0, shy = 0;
int choice;

// Sheared coordinates
float nx1, ny1, nx2, ny2, nx3, ny3;
void init(void)
{
    glClearColor(0.0,0.0,0.0,0.0); //GLfloat red,green,blue,alpha initial value 0 alpha values used by
glclear to clear the color buffers
    glMatrixMode(GL_PROJECTION); // To specify which matrix is the current matrix & projection applies
subsequent matrix to projection matrix stack
    glLoadIdentity();
    glOrtho(0.0, 400.0, 0.0, 400.0, -1.0, 1.0);
    //gluOrtho2D(0.0,300.0,0.0,300.0); // Orthographic representation; multiply the current matrix by an
orthographic matrix 2D= left right,bottom,top equivalent near=-1,far=1
}

void Draw()
{
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw Original Triangle (Blue)
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
        glVertex2f(x1, y1);
        glVertex2f(x2, y2);
        glVertex2f(x3, y3);
    glEnd();

    // Draw Sheared Triangle (Red)
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_TRIANGLES);
        glVertex2f(nx1, ny1);
        glVertex2f(nx2, ny2);
        glVertex2f(nx3, ny3);
    glEnd();

    glutSwapBuffers();
}

int main(int argc,char **argv){
    printf("Choose Reflection Type:\n");
    printf("Choose Shearing Type:\n");
    printf("1. Shearing along X-axis (Horizontal Shear)\n");
    printf("2. Shearing along Y-axis (Vertical Shear)\n");
    printf("Enter your choice (1 or 2): ");
    scanf("%d", &choice);

    if(choice == 1)

```

```

{
    printf("Enter shear factor along X-axis (shx): ");
    scanf("%f", &shx);

    // Horizontal Shear Formula
    nx1 = x1 + shx * y1;
    ny1 = y1;

    nx2 = x2 + shx * y2;
    ny2 = y2;

    nx3 = x3 + shx * y3;
    ny3 = y3;
}
else if(choice == 2)
{
    printf("Enter shear factor along Y-axis (shy): ");
    scanf("%f", &shy);

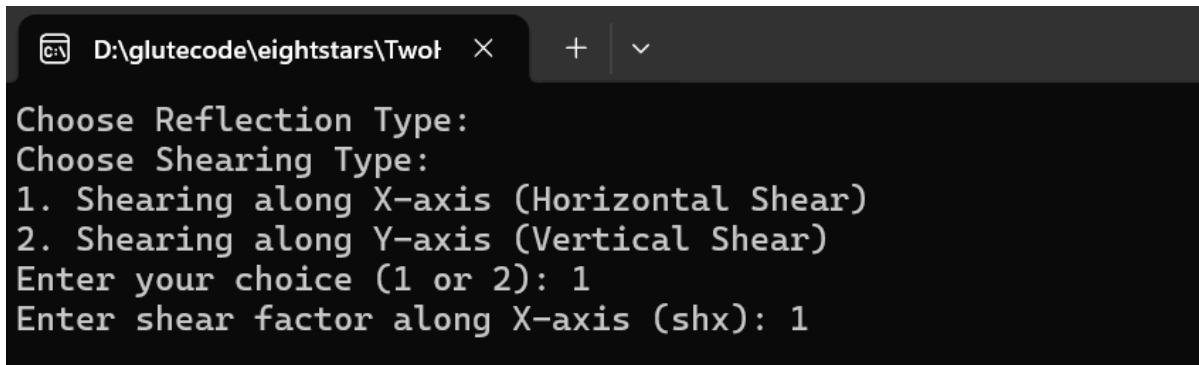
    // Vertical Shear Formula
    nx1 = x1;
    ny1 = y1 + shy * x1;

    nx2 = x2;
    ny2 = y2 + shy * x2;

    nx3 = x3;
    ny3 = y3 + shy * x3;
}
else
{
    printf("Invalid choice! Program will exit.");
    return 0;
}
glutInit(&argc,argv);
glutInitDisplayMode ( GLUT_RGB | GLUT_DOUBLE );
glutInitWindowPosition(0,0);
glutInitWindowSize(500,500);
glutCreateWindow("Lab Final");
init();
glutDisplayFunc(Draw);
glutMainLoop();
return 0;
}

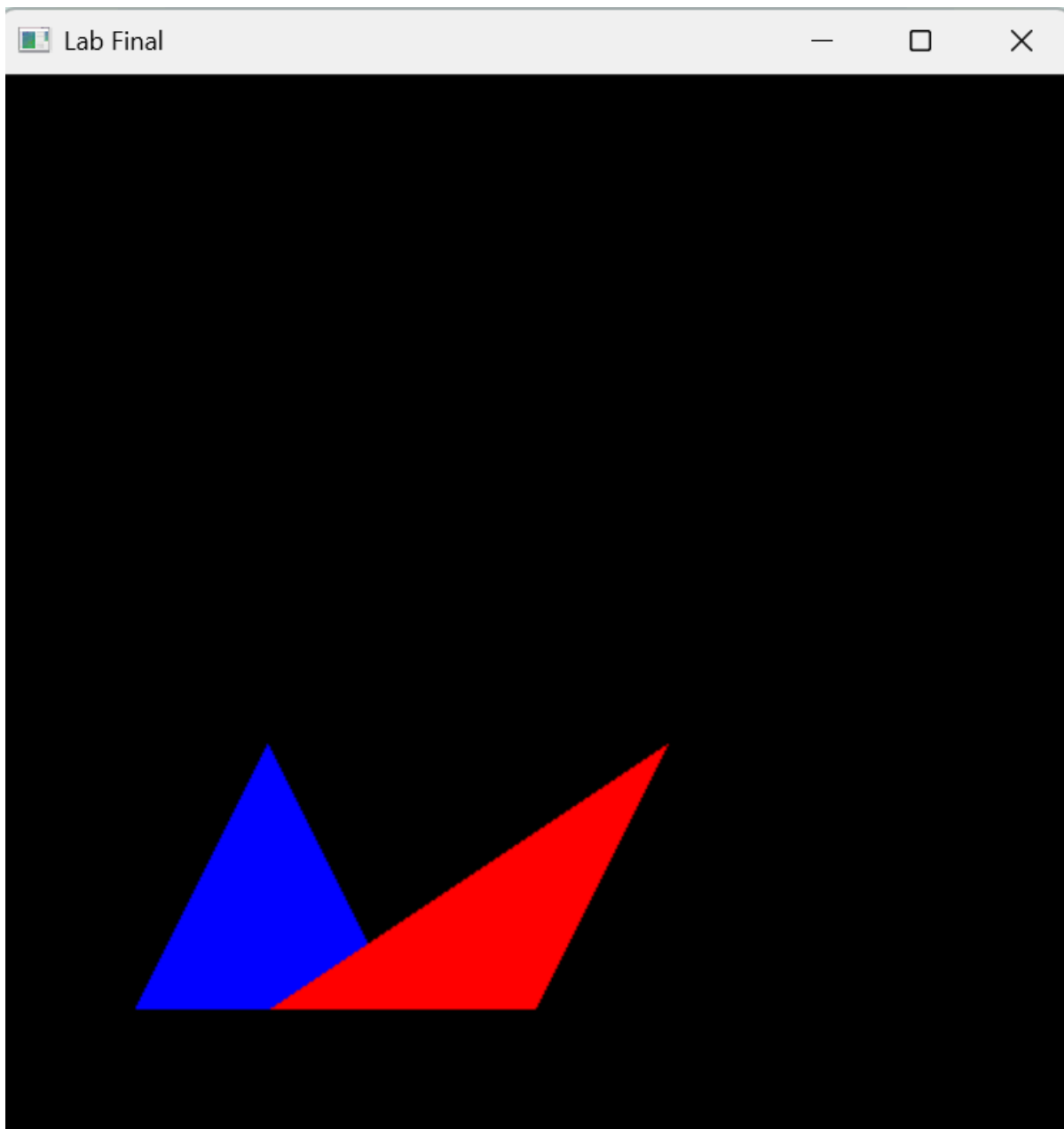
```

Input:



```
D:\glutecode\eightstars\TwoT  X  +  v
Choose Reflection Type:
Choose Shearing Type:
1. Shearing along X-axis (Horizontal Shear)
2. Shearing along Y-axis (Vertical Shear)
Enter your choice (1 or 2): 1
Enter shear factor along X-axis (shx): 1
```

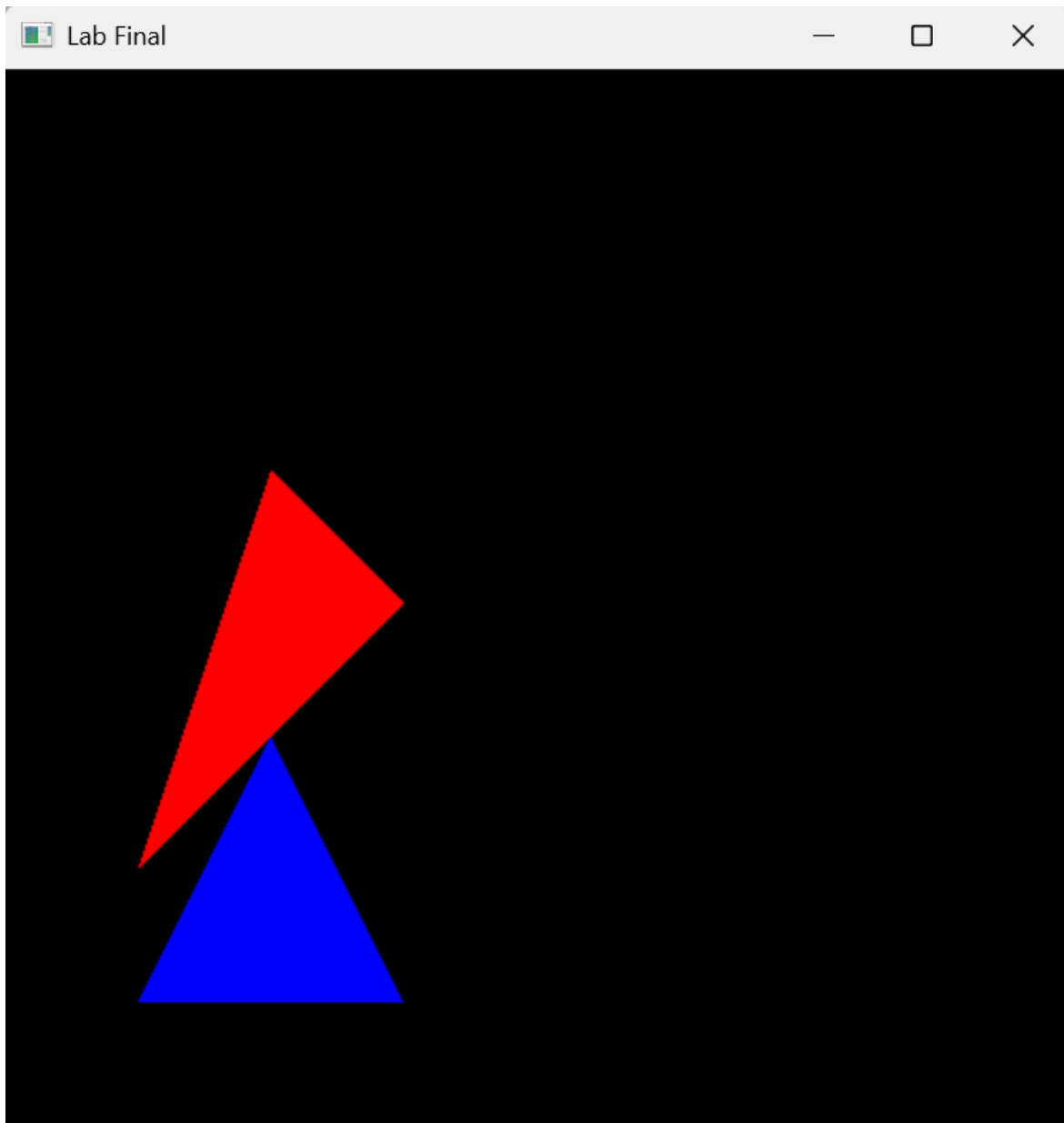
Output:



Input:

```
D:\glutecode\eightstars\Two1 x + v
Choose Reflection Type:
Choose Shearing Type:
1. Shearing along X-axis (Horizontal Shear)
2. Shearing along Y-axis (Vertical Shear)
Enter your choice (1 or 2): 2
Enter shear factor along Y-axis (shy): 1
```

Output:



5.5 Rotation:

```
//Shafin Ahmed 232-15-184 65-M1
//Rotation
#include <GL/glut.h>
#include <stdio.h>
#include <GL/gl.h>
GLfloat spin=0.00;
void init(void)
{
```



```

    glClearColor(0.0,0.0,0.0,0.0); //GLfloat red,green,blue,alpha initial value 0 alpha values used by
    glClear to clear the color buffers
    glMatrixMode(GL_PROJECTION); // To specify which matrix is the current matrix & projection
    applies subsequent matrix to projection matrix stack
    glLoadIdentity();
    glOrtho(-100.0, 100.0, -100.0, 100.0, -1.0, 1.0);
    //gluOrtho2D(0.0,300.0,0.0,300.0); // Orthographic representation; multiply the current matrix by an
    orthographic matrix 2D= left right,bottom,top equivalent near=-1,far=1
}
void Draw()
{
    glClear(GL_COLOR_BUFFER_BIT);

    glPushMatrix();
    glColor3f( 1 ,0, 0);
    glRotatef(spin, 0.0, 0.0, 1.0);
    glRectf(-75,-75,75,75);
    glPopMatrix();

    glPushMatrix();
    glColor3f( 0 ,1, 0);
    glRotatef(spin, 0.0, 0.0, 1.0);
    glRectf(-45,-45,45,45);
    glPopMatrix();

    glPushMatrix();
    glColor3f( 0 ,0, 1);
    glRotatef(spin, 0.0, 0.0, 1.0);
    glBegin(GL_POLYGON);
    glVertex2f(0.00,20.00);
    glVertex2f(-19.02,6.18);
    glVertex2f(-11.76,-16.18);
    glVertex2f(11.76,-16.18);
    glVertex2f(19.02,6.18);
    glEnd();
    glPopMatrix();

    glutSwapBuffers();
}
void spinDisplay_left(void)
{
    spin = spin + 10;
    glutPostRedisplay();
}
void spinDisplay_right(void)
{
    spin = spin - 10;
    glutPostRedisplay();
}
void my_keyboard(unsigned char key,int x, int y)
{
    switch (key) {
        case 'l':
            spinDisplay_left();
    }
}

```

```
        break;
    case 'r':
        spinDisplay_right();
        break;
    case 's':
        glutIdleFunc(NULL);
        break;
    default:
        break;
}
}
int main(int argc, char **argv){
    glutInit(&argc, argv);
    glutInitDisplayMode ( GLUT_RGB | GLUT_DOUBLE );
    glutInitWindowPosition(0,0);
    glutInitWindowSize(500,500);
    glutCreateWindow("Lab Final");
    init();
    glutDisplayFunc(Draw);
    glutKeyboardFunc(my_keyboard);
    glutMainLoop();
    return 0;
}
```

Output:

